Treći popravni test iz Algoritama i struktura podataka

Prvi praktični test održan 06.03.2026.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
   a. Vodite računa da imate sve spremno što vam je neophodno.
   b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test2_GrupaB_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Implementacija DFSVisit metode Depth-First-Search algoritma*

Napisati implementaciju DFSVisit metode istoimenog algoritma.

a) Napisati algoritam.
b) U *Main* metodi kreirati proizvoljan graf. Obezbediti način za testiranje i proveru nove funkcionalnosti, odnosno poziv ***DFS*** metode.

Dodatne napomene za rešavanje zadatka:

- U template-u je metoda ***private void DFSVisit(LinkedList.Node node, int nodeIndex)*** koja je osnovna metoda za rekurzivno izvršavanje DFS algoritma.

Zadatak 2 – *Čišćenje duplikata (Keep First Occurrence)*

Napisati metodu koja iz prosleđenog stringa uklanja sve **naknadne** pojave karaktera koji su se već pojavili, ostavljajući samo njihovu **prvu pojavu**. Rezultat ispisati u konzoli. Algoritam treba da bude vremenske kompleksnosti O(n) i *case-insensitive*.

Dodatni zahtevi za rešavanje zadatka:

- Koristiti Hash tabelu iz šablona za čuvanje broja pojavljivanja svakog karaktera.
- Implementaciju pisati u metodi: ***public static void keepFirstOccurrence (string str).***
- Vremenska kompleksnost treba da bude **O(n).**
- Ignorisati razliku između malih i velikih slova.

Primer:

Input 1: *str = „Programiranje"*

Output 1: *„Progaminje" (Karakteri „r" i „a" se ponavljaju)*

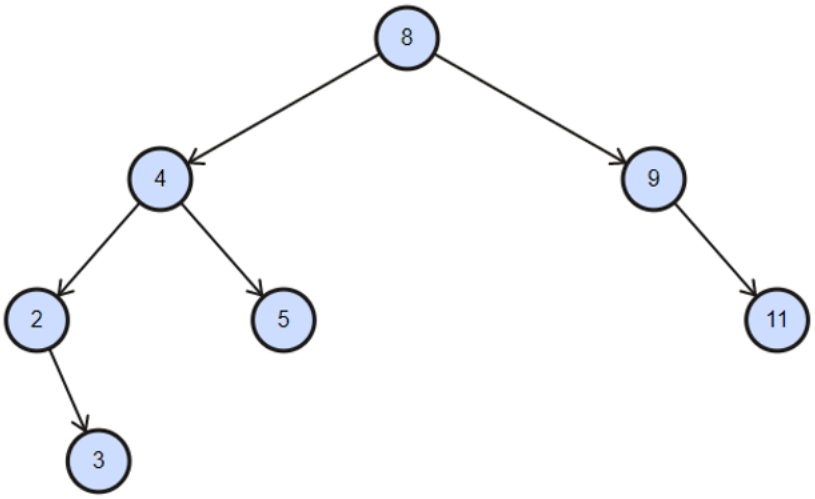
Zadatak 3 – *Brojanje čvorova na specifičnom nivou N*

U okviru klase BinarySearchTree napisati metodu koja broji koliko se ukupno čvorova nalazi na tačno određenom nivou (dubini) N, gde je koren stabla na nivou 0.

Dodatni zahtevi za rešavanje zadatka:

1. Javna metoda: ***public int countNodesAtLevel (int targetLevel).***
2. Privatna metoda: ***private int countNodesAtLevel (Node current, int currentLevel, int targetLevel).***
3. Podrazumeva se da stablo ima barem dva čvora.
4. Algoritam **OBAVEZNO** pisati u okviru klase ***BinarySearchTree***

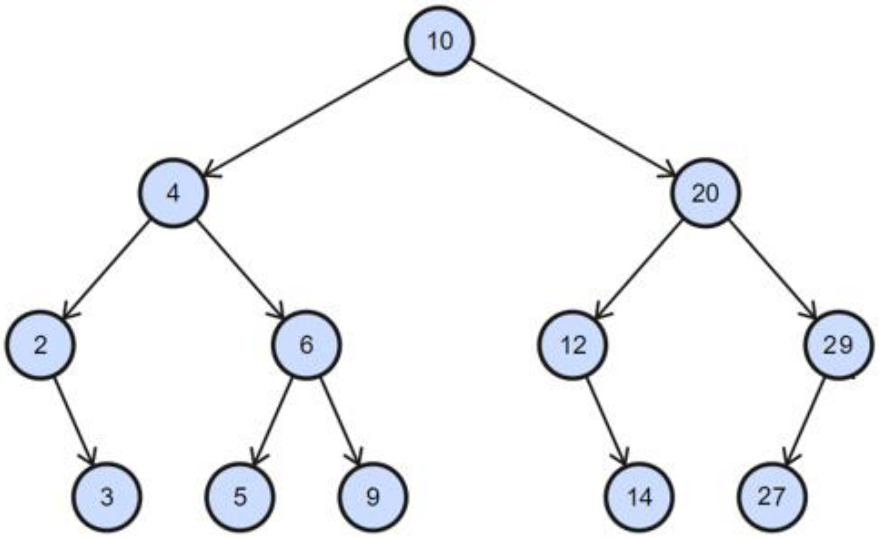
Primeri:



Slika 1 - Primer BST-a

Input 1: *targetLevel = 3*

Output 1: *1 („3“ se nalazi na toj dubini)*



Slika 2 - Primer BST-a

Input 2: *targetLevel = 2*

Output 2: *4 („2“, „6“, „12“ i „29“)*